



Desenvolvimento Web

Profª Sabrina B. Moreira



Aprendendo CSS 3

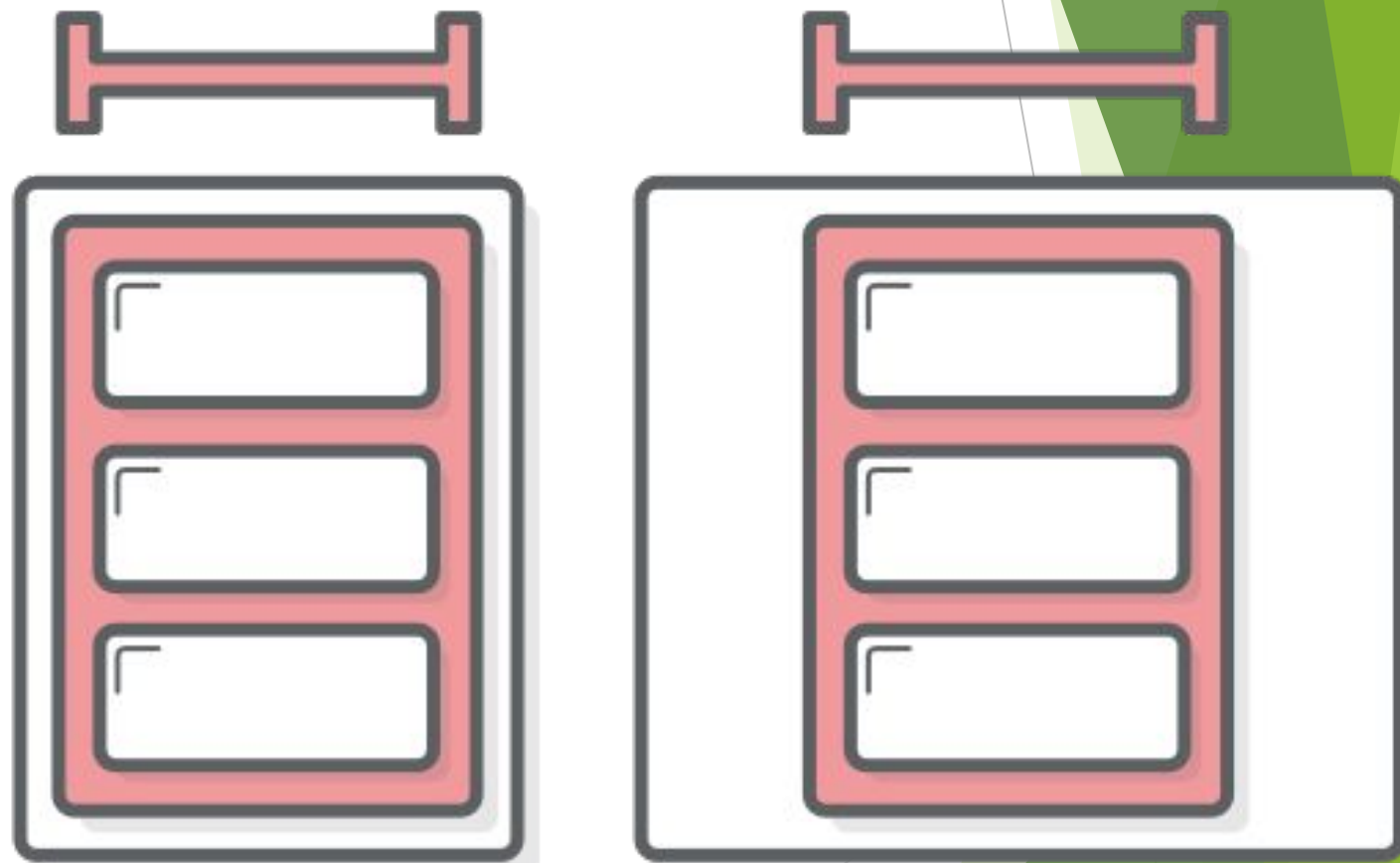
Aula 11.1: responsividade e medidas relativas

CSS: abordagens para estruturação de páginas

- ❖ No desenvolvimento web, a forma como um layout se comporta em diferentes tamanhos de tela é essencial para garantir uma boa experiência ao usuário.
- ❖ Existem diferentes abordagens para **estruturar páginas**, entre elas o layout fixo e o layout fluido, que se diferenciam principalmente pela **maneira como utilizam o espaço disponível na tela** e se adaptam (ou não) a diferentes dispositivos.

CSS: layout fixo

- ❖ Um layout fixo mantém sempre a mesma estrutura, independentemente da largura da tela.



CSS: layout fixo



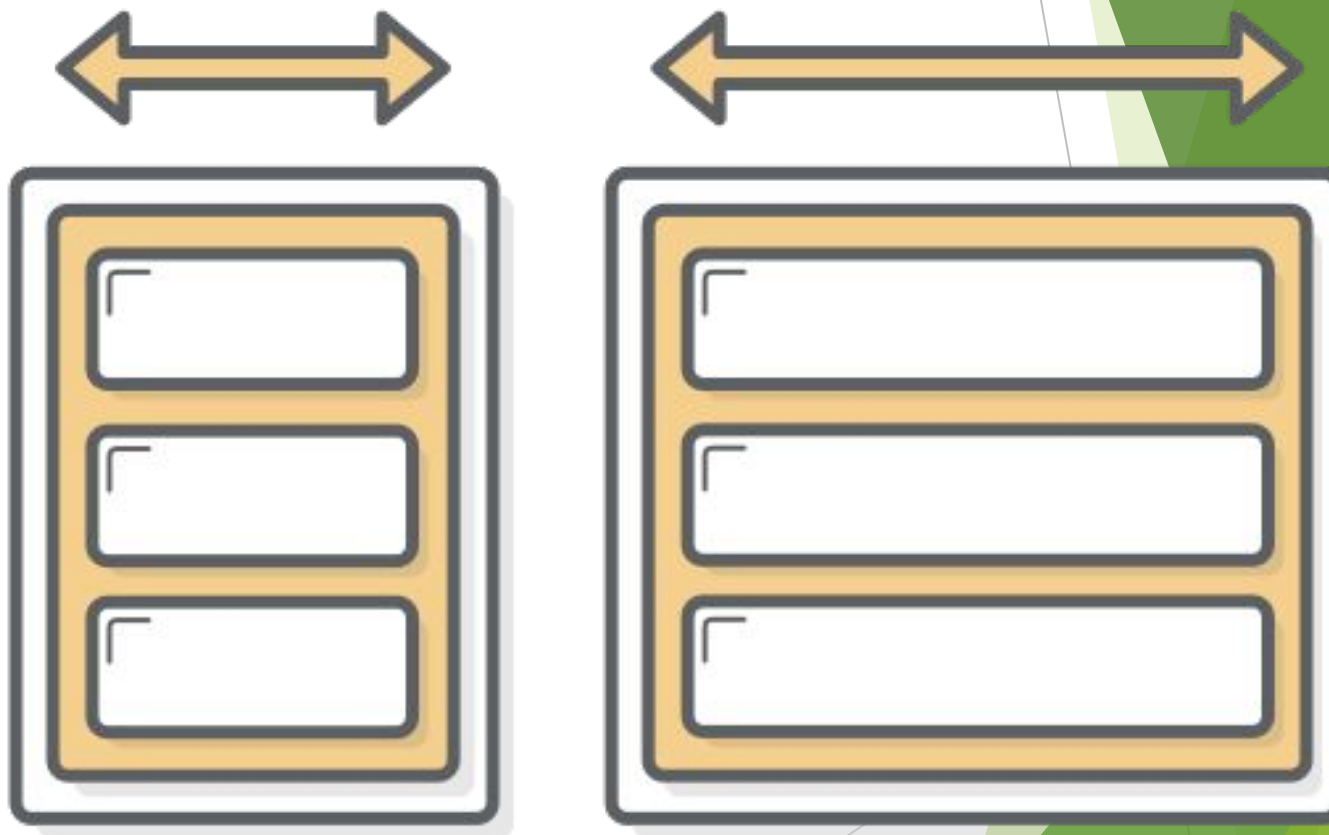
```
main {  
  max-width: 800px;  
  margin: auto;  
}
```

main 864 × 1617.56

Fonte: autora, 2026

CSS: layout fluido

- ❖ Já o layout fluido se modifica para preencher a tela e se tornar visualmente mais agradável.



Fonte: Pires, 2026

CSS: layout fluido



Fonte: autora, 2026

CSS: estratégias de adaptação de páginas

- ❖ Além da forma como os layouts são estruturados, também existem **diferentes estratégias de adaptação das páginas** para múltiplos dispositivos.
- ❖ Essas abordagens definem como um site se comporta ao ser acessado em telas de diferentes tamanhos, podendo alternar entre versões específicas ou se ajustar de maneira contínua.

CSS: design adaptativo

- ❖ Cria vários layouts fixos para tamanhos específicos de tela (ex: 320px, 768px, 1024px), carregando a versão mais adequada para o dispositivo.

Usa:

- ❖ Usa vários layouts fixos que alternam entre os tamanhos de tela
- ❖ Media queries (separando as versões)

O site **troca de versão**, não se ajusta suavemente



CSS: design responsivo

- ❖ O layout se adapta **automaticamente** a diferentes tamanhos de tela, garantindo uma boa experiência em **qualquer dispositivo**.

Usa:

- ❖ Usa layout fluido
- ❖ Media queries (ajuste contínuo)

Ele se adapta continuamente, sem “pular” entre versões



Fonte: Pires, 2026

CSS: design responsivo

- ❖ Telas construídas com **grade fluida** (Fluid Grid Layout): Utiliza unidades relativas (% , rem, vw) em vez de pixels fixos.
- ❖ **Media queries**: criação de regras de estilo específicos para cada largura de tela.
- ❖ **Imagens flexíveis**: imagens dimensionáveis ou alternância entre diferentes imagens.
- ❖ **Tipografia adaptativa**: uso de unidades relativas para que o texto se ajuste ao tamanho da tela.

ADAPTIVE

Generates templates which are optimized and unique for every device class



RESPONSIVE

Universal design which reflows across displays



CSS Unidades de medidas fixas

CSS: unidades de medidas fixas

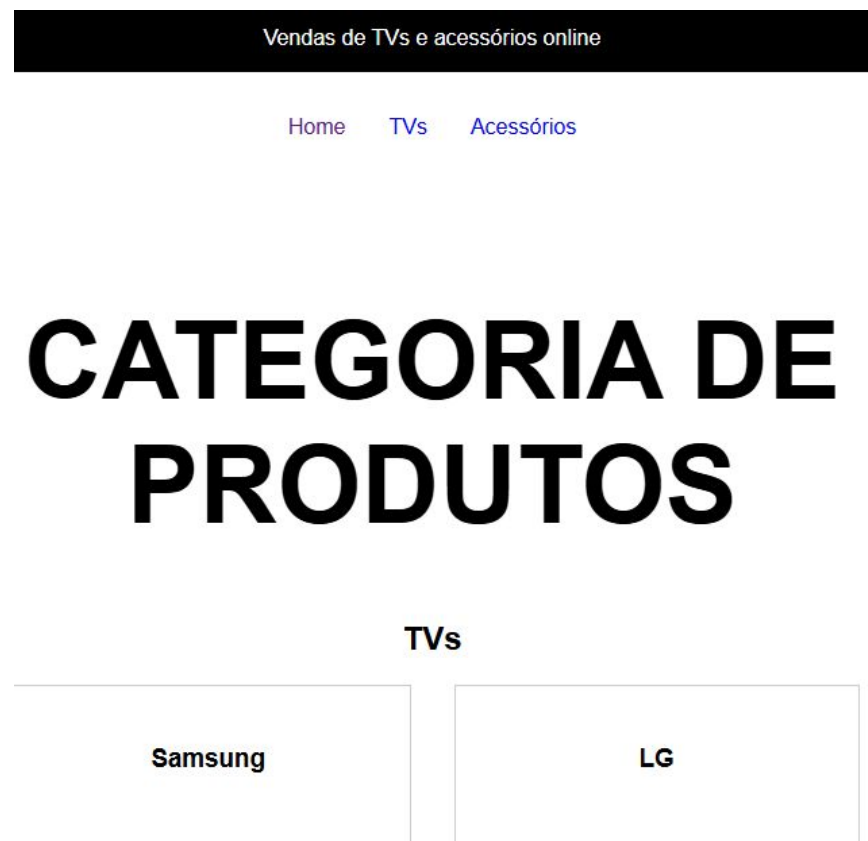
- ❖ Quando você está começando no CSS, é comum usar unidades fixas como px (pixels), porque elas parecem mais simples e previsíveis.
- ❖ Por exemplo, definir um tamanho fixo para fonte de texto.

```
h1 {  
  font-size: 80px;  
}
```

Fonte: autora, 2026

CSS: unidades de medidas fixas

- ❖ O problema é que isso **não se adapta bem a diferentes tamanhos de tela.**
- ❖ O que funciona no computador pode ficar **estranho** ou até **quebrado** no celular.



CSS: unidades de medidas fixas x flexíveis

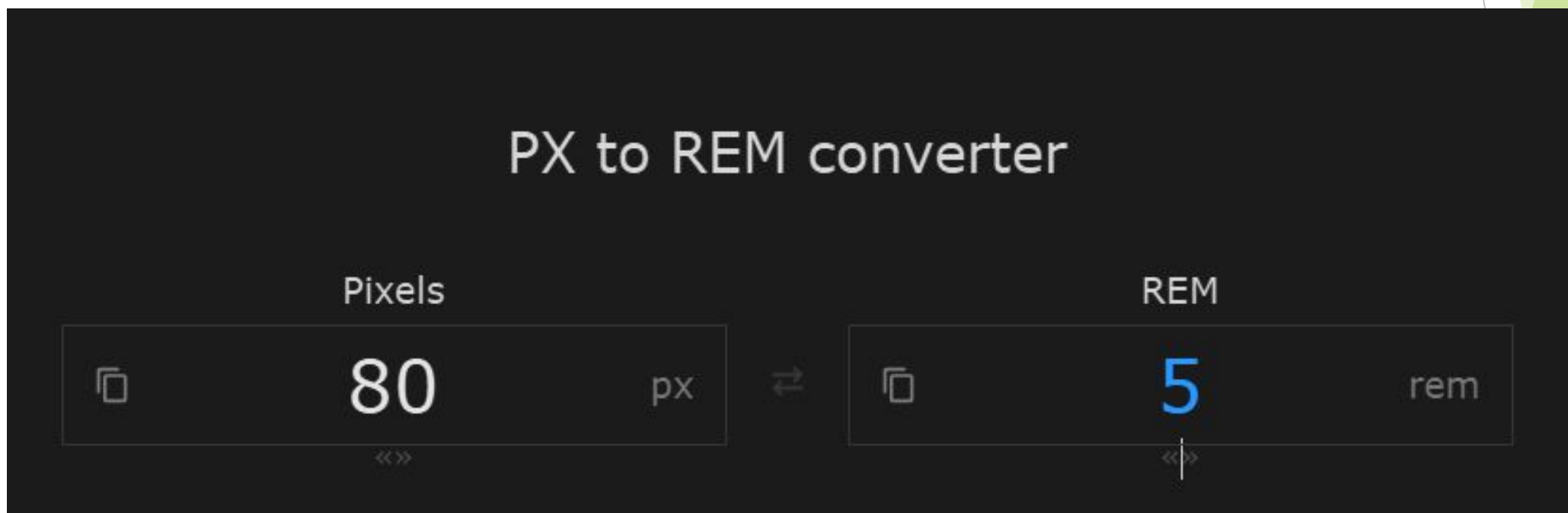
- ❖ Unidades absolutas são fixas e não se adaptam à tela, sendo úteis para **controle preciso**, mas pouco responsivas.
- ❖ Unidades relativas se adaptam ao contexto ou à viewport, sendo ideais para **layouts responsivos**.

ABSOLUTOS	RELATIVOS
px	rem
pt	em
pc	vw
in	vh
cm	vmin
mm	vmax
	ex
	ch

CSS Unidades de medidas flexíveis

CSS: unidades de medidas flexíveis

- ❖ As unidades flexíveis (ou relativas) permitem que os elementos **se ajustem automaticamente** ao tamanho da tela ou ao espaço disponível.



CSS: unidades de medidas flexíveis

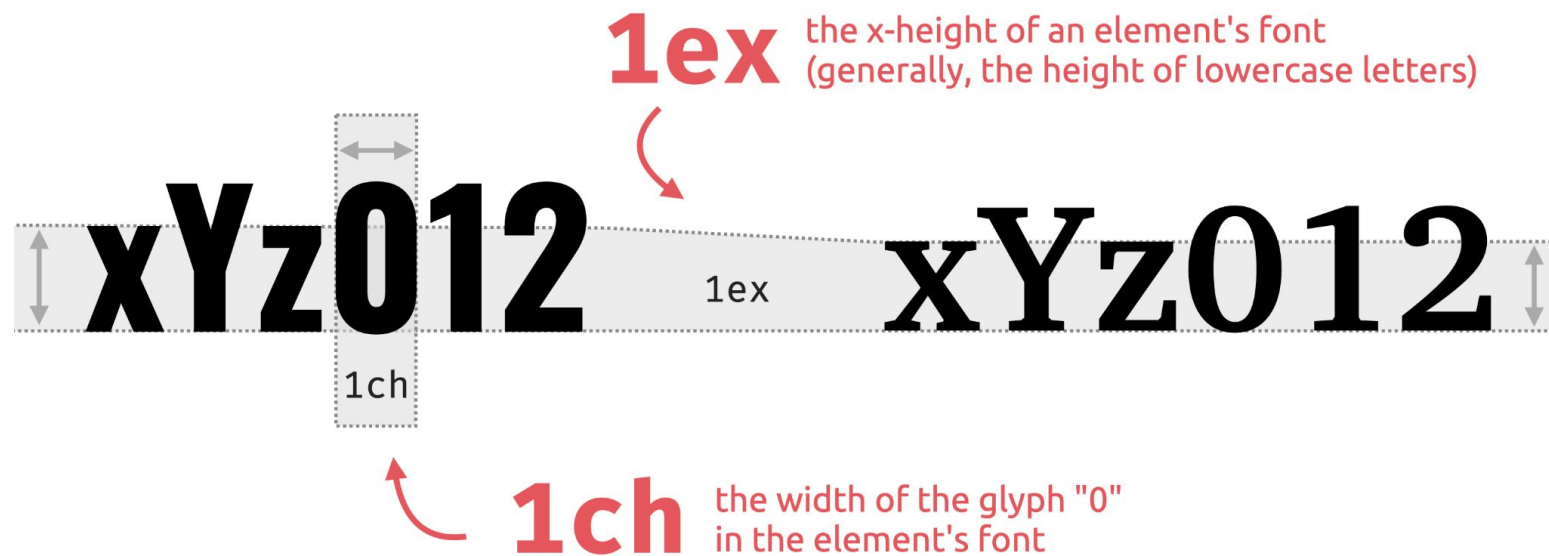
- ❖ Desde do CSS 1 (1996), já tínhamos algumas medidas relativas, como *em* e %, que apresentavam forte dependência entre os elementos para exibição destas medidas.



CSS: unidades de medidas flexíveis

- ❖ Na versão 2 (1998), entraram a *ex* (altura do x da fonte) e *ch* (largura do caracter 0) mas por variar muito de acordo com a **fonte** não é usado por ser difícil prever seu comportamento.

The ex & ch unit in CSS



#devsheets by @stefanjudis

Fonte: Judis, 2020

CSS: unidades de medidas flexíveis

Já o CSS 3 (2010) com a popularização dos celulares, surgiram unidades de medidas mais modernas, como:

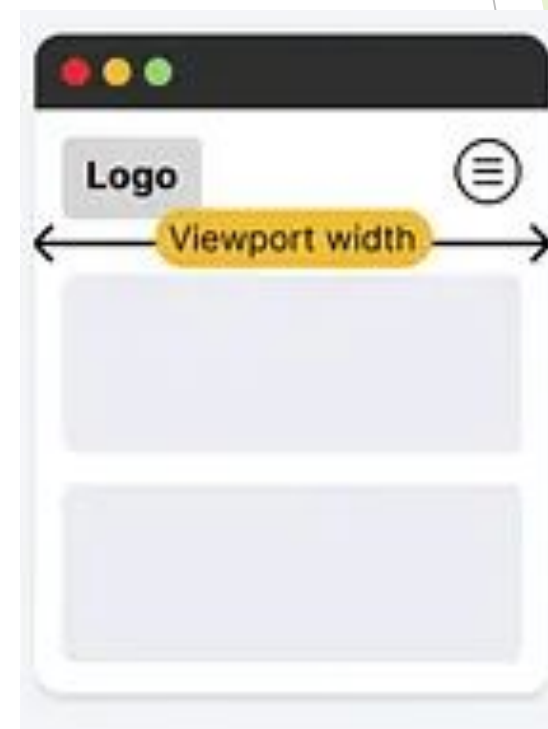
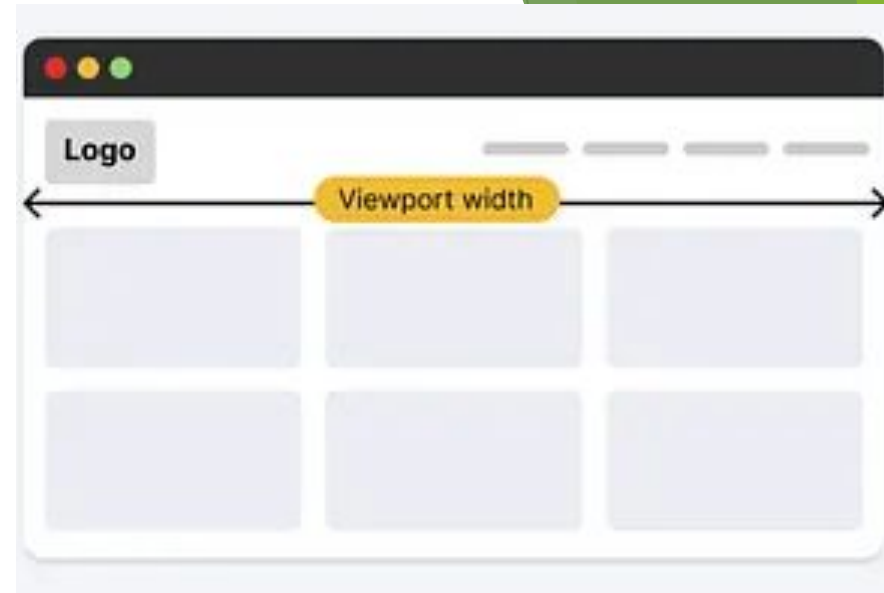
- ❖ *rem* (baseado no root -> html)
- ❖ *Viewport units* (vw, vh, vmin e vmax)



CSS: Viewport

É a área visível da página no navegador, ou seja, o que o usuário realmente **enxerga na tela**.

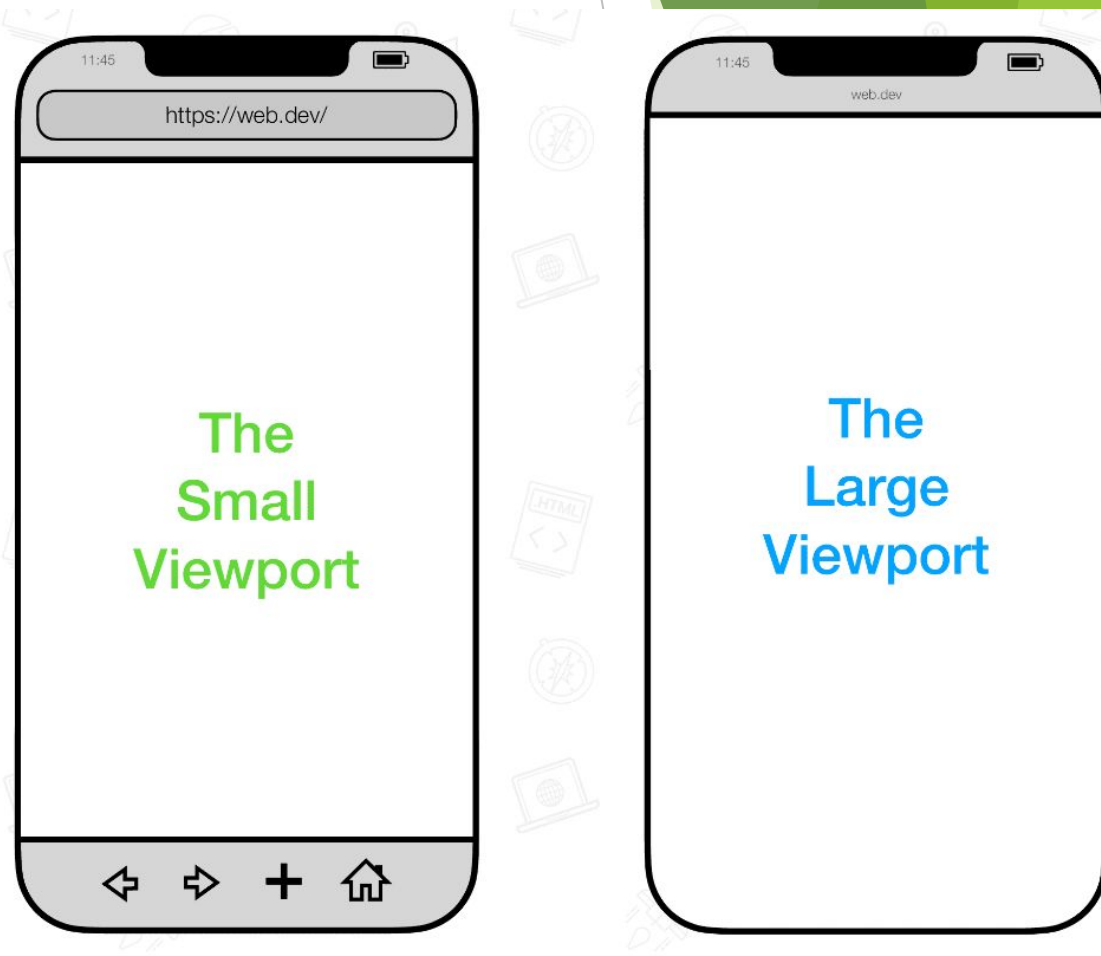
Exemplo, o usuário está no celular e a página pode ser gigante (scroll infinito) mas **ele vê só um pedaço**, esse pedaço visível é **viewport atual**.



CSS: unidades de medidas flexíveis

Após 2020, surgiram novos problemas nas medidas responsivas pois a barra de url mudava a altura da tela com o uso das medidas relativas.

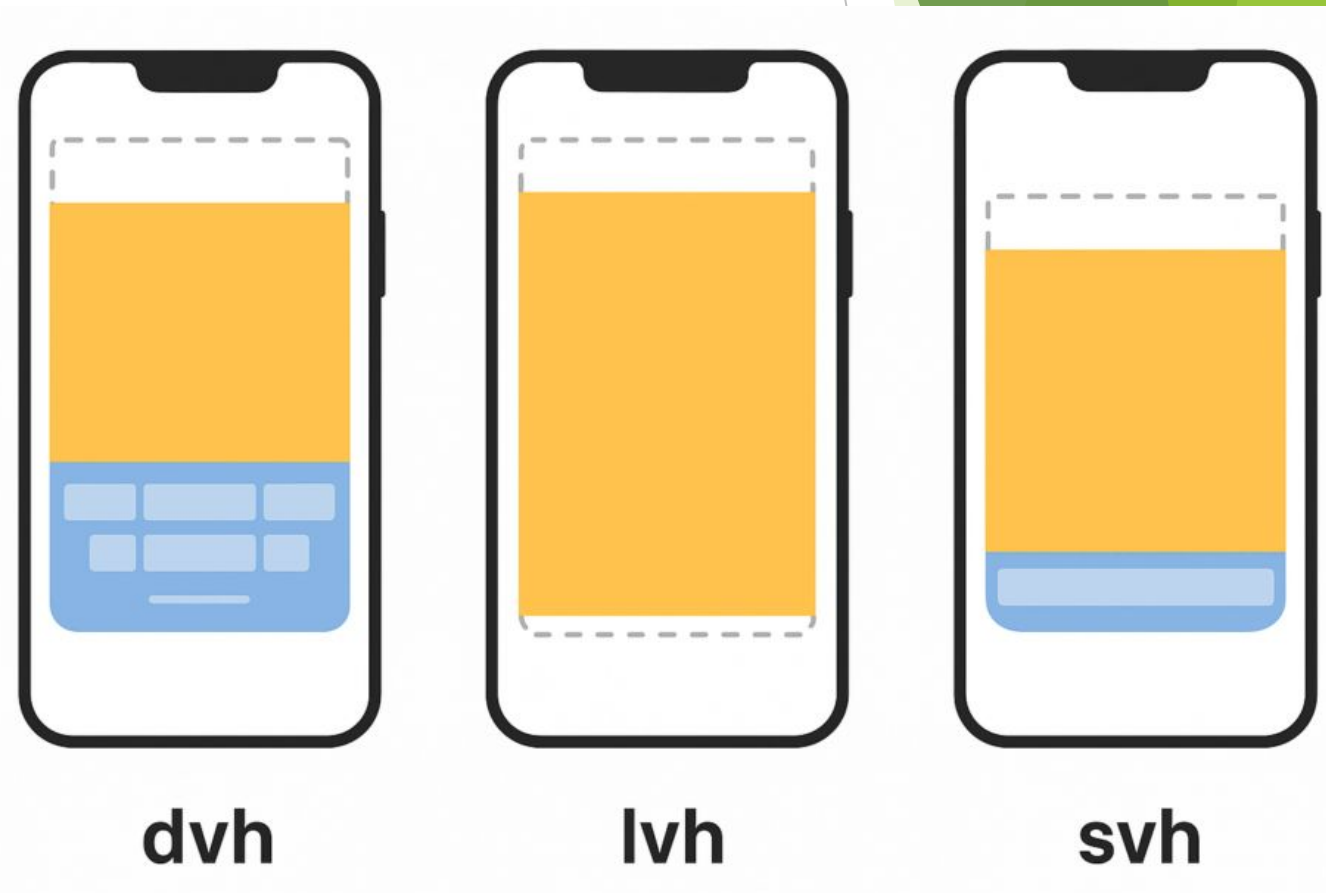
- ❖ *O viewport não é mais fixo, ele pode crescer e diminuir conforme o navegador mostra/esconde elementos.*



CSS: unidades de medidas flexíveis

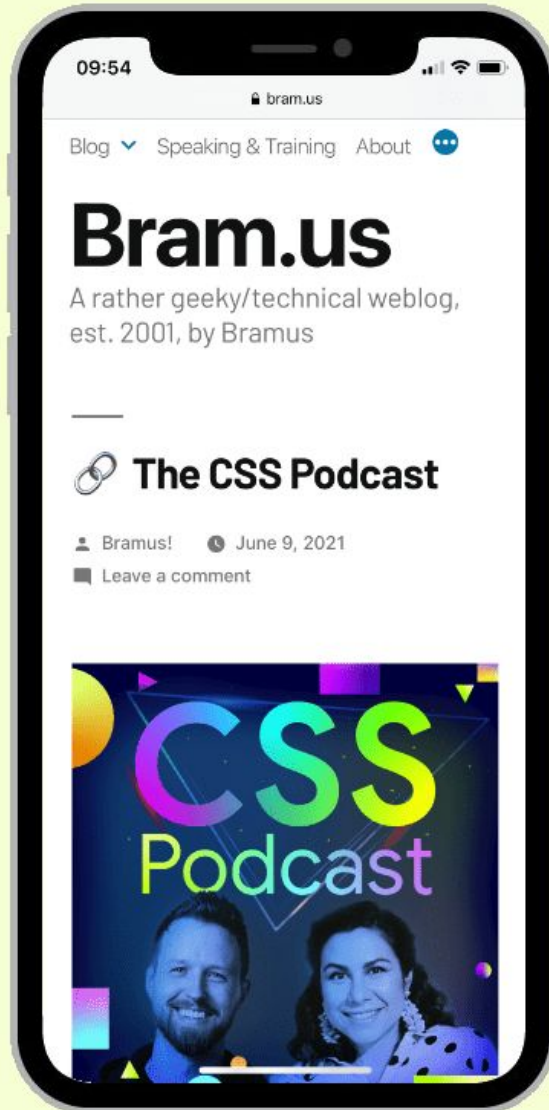
E com isso surgiram novas medidas para oferecer maior controle:

- ❖ ***dvh***: altura dinâmica real vista naquele momento
- ❖ ***lvh***: maior altura possível
- ❖ ***svh***: menor altura possível mínima garantida



LARGE VIEWPORT

The viewport sized assuming any UA interfaces (such as the address bar) that are dynamically expanded and retracted to be **retracted**

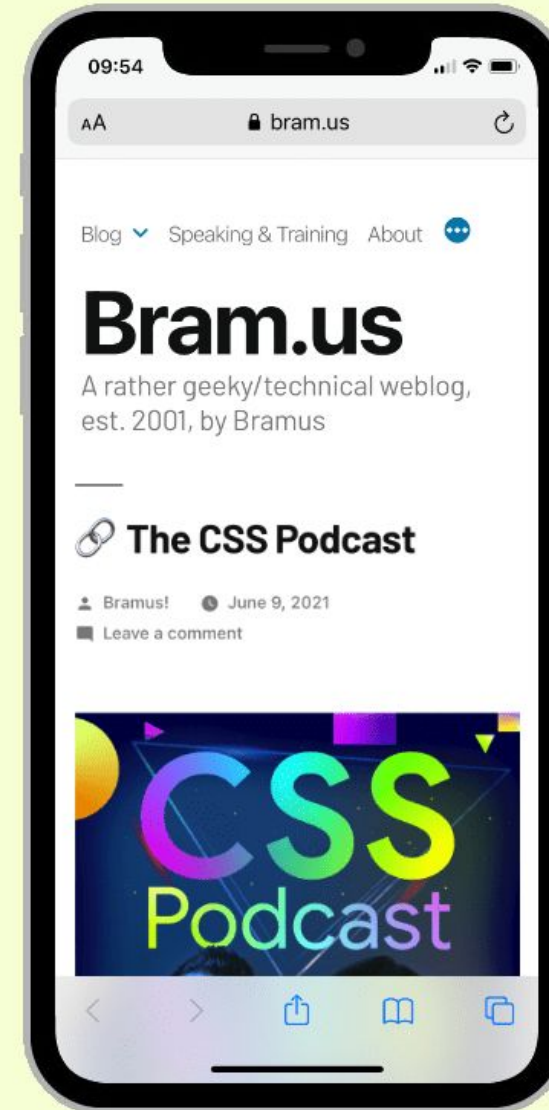


100lvh

100svh

SMALL VIEWPORT

The viewport sized assuming any UA interfaces (such as the address bar) that are dynamically expanded and retracted to be **expanded**

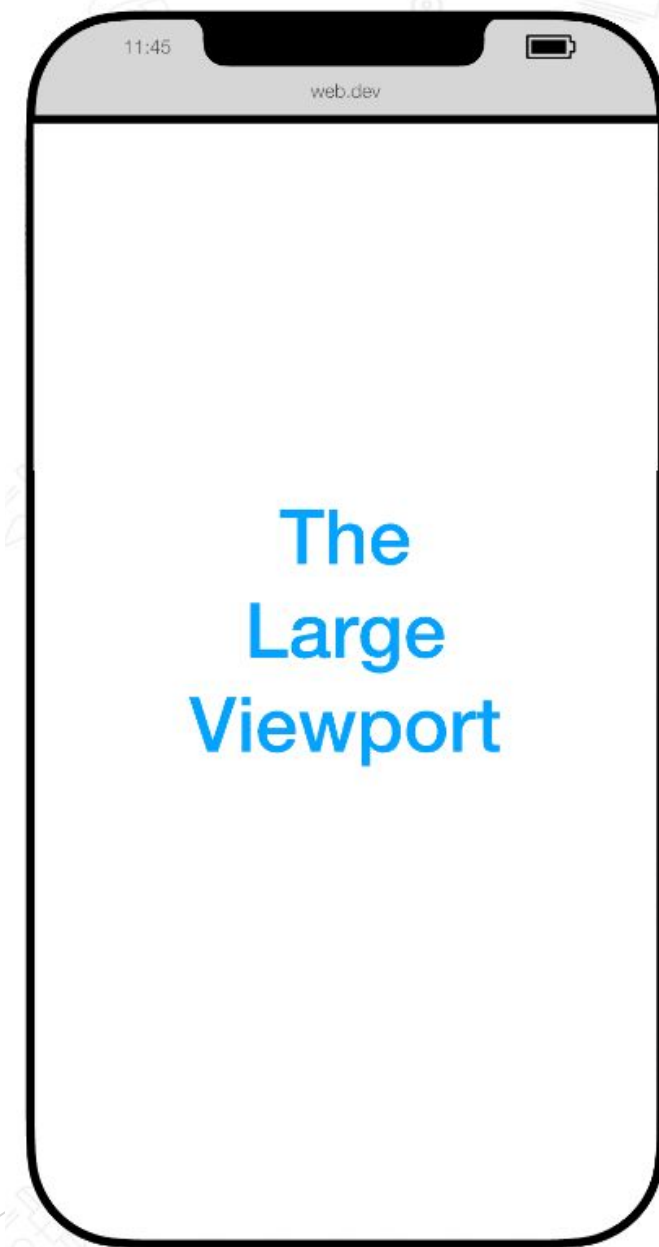


CSS: unidades de medidas flexíveis

- ❖ As unidades que representam a **janela de visualização grande** têm o prefixo **lv**.

As unidades são:

- ❖ **lvw**: largura máxima da Viewport
- ❖ **lvh**: altura máxima da Viewport
- ❖ **lvi**: eixo inline máximo da Viewport
- ❖ **lvb**: eixo block máximo da Viewport
- ❖ **lvmin**: menor valor entre o lvw e lvh
- ❖ **lvmax**: maior valor entre o lvw e lvh



CSS: unidades de medidas flexíveis

- ❖ As unidades que representam a **janela de visualização pequena** têm o prefixo **sv**.

As unidades são:

- ❖ **svw**: menor largura que a Viewport pode ter
- ❖ **svh**: menor altura que a Viewport pode ter
- ❖ **svi**: eixo inline mínimo da Viewport
- ❖ **svb**: eixo block mínimo da Viewport
- ❖ **svmin**: menor valor entre o svw e svh
- ❖ **svmax**: maior valor entre o svw e svh



CSS tipos de medidas flexíveis

CSS: unidades de medidas flexíveis %

- ❖ % (porcentagem): Define o tamanho em relação ao elemento pai.

Exemplo: uma fonte que apresenta o dobro do tamanho do elemento pai.

```
main {  
  font-size: 100%; /* 16px */  
}  
  
.big-title {  
  font-size: 200%; /* 32px */  
}
```

**CATEGORIA DE
PRODUTOS**

TVs

As melhores TVs você encontra
aqui

Samsung



CSS: unidades de medidas flexíveis %

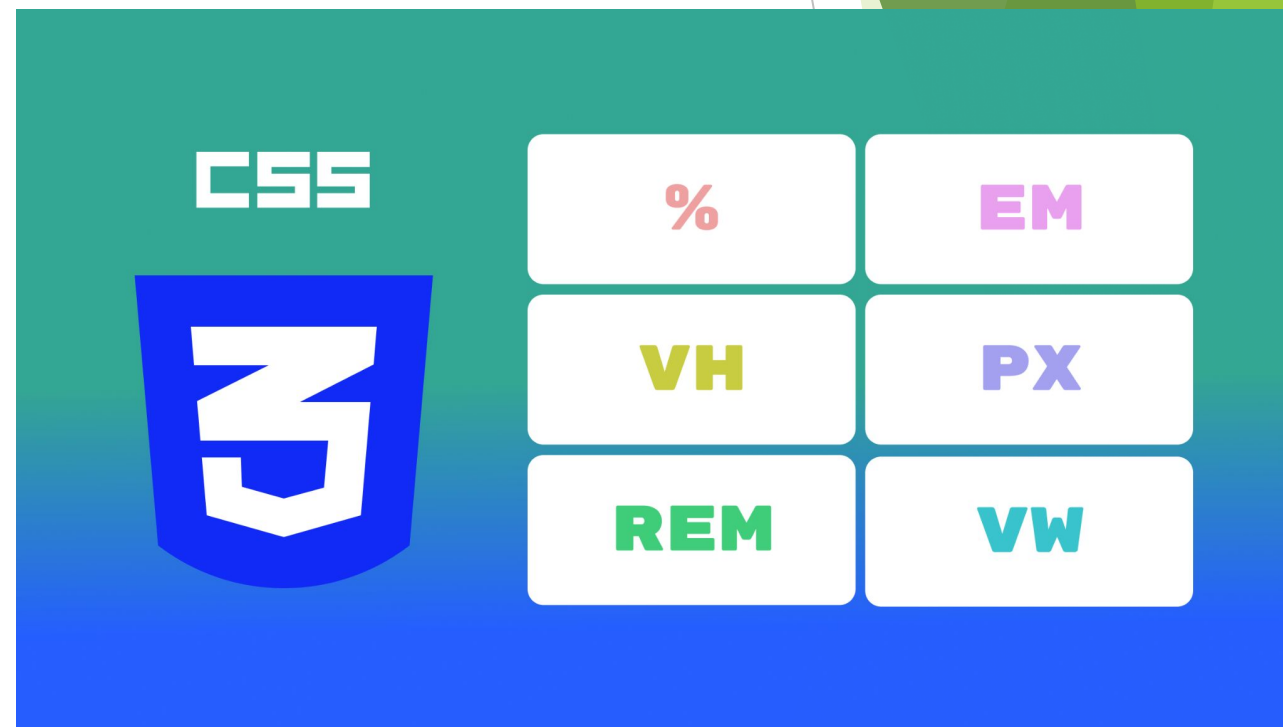
Por que a % é tão usada?

- ❖ Se adapta ao elemento pai
- ❖ Essencial para layouts fluidos
- ❖ Funciona bem com Grid/Flexbox

É a medida relativa **mais utilizada** para definir elementos em layouts.

CSS: unidades de medidas flexíveis em

- ❖ **em (element):** é uma unidade relativa ao tamanho da fonte do elemento pai.
- ❖ Seu valor é acumulativo.



Fonte: Braide, 2024

CSS: unidades de medidas flexíveis em

❖ Exemplo em (element)

Exemplo:

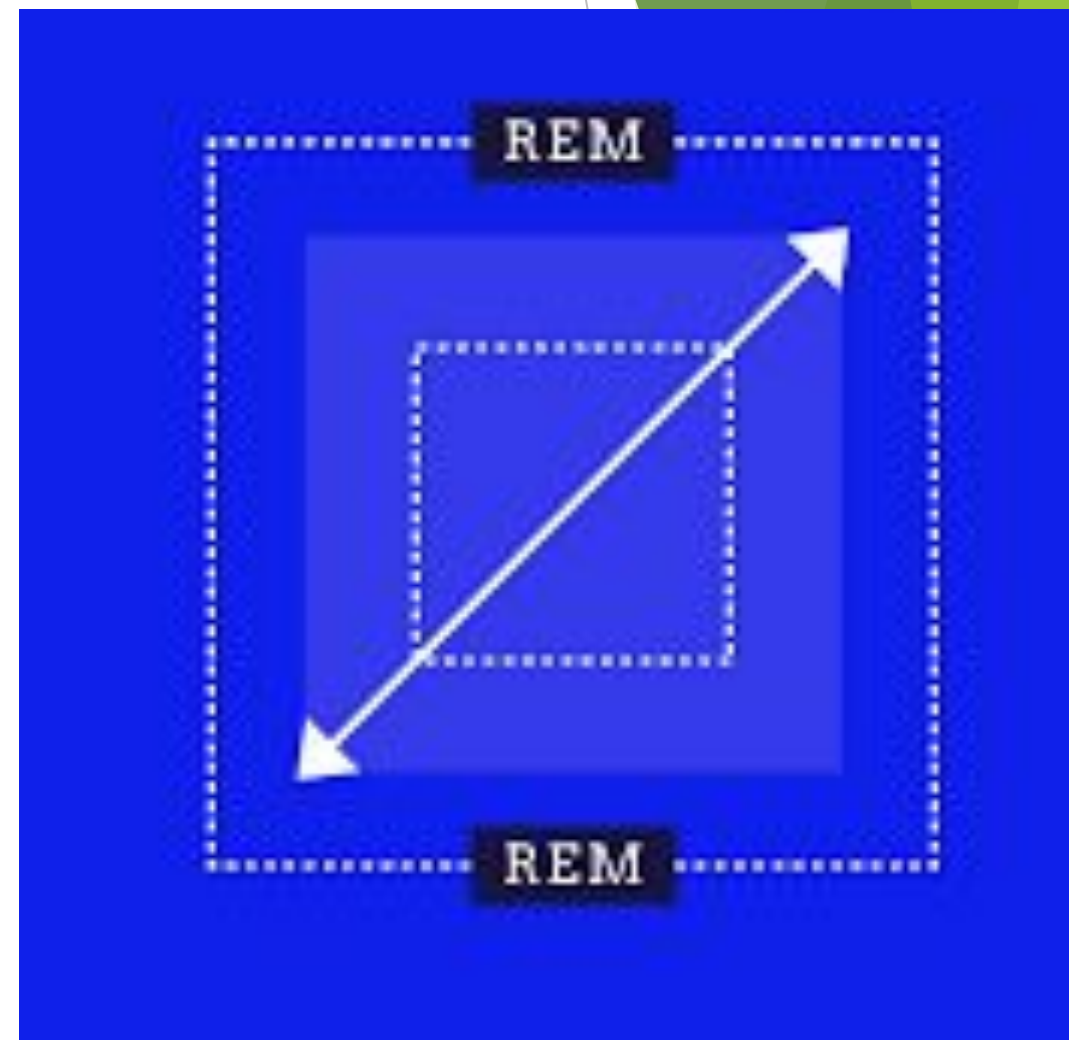
- ❖ O valor é calculado com base no font-size do elemento pai e pode se **acumular** em elementos aninhados.

```
html {  
  font-size: 12px;  
}  
  
main {  
  font-size: 2em;  
  /* 24px */  
}  
  
.big-title {  
  font-size: 4em;  
  /* 96px */  
}
```



CSS: unidades de medidas flexíveis rem

- ❖ **rem (root element):** é relativo ao tamanho da fonte do elemento raiz (html).
- ❖ O elemento raiz é o elemento mais alto da árvore do documento, ou seja, o primeiro que **envolve todos** os outros.



CSS: unidades de medidas flexíveis rem

❖ Exemplo (root element)

Exemplo:

- ❖ Define o tamanho com base no valor do font-size do elemento raiz (html), independentemente do elemento pai.

```
html {  
  font-size: 12px;  
}  
  
main {  
  font-size: 2rem;  
}  
  
.big-title {  
  font-size: 4rem;  
  /* 48px */  
}
```

CATEGORIA DE PRODUTOS

TVs

TVs disponíveis

Confira preços e condições de parcelamento

Samsung



TV Samsung Smart

Preço: R\$ 2.000,00

Parcelamento: R\$ 200,00 em 10x

CSS: unidades de medidas flexíveis rem

Por que a **rem** é tão usada?

- ❖ Previsível (não sofre cascata como em)
- ❖ Fácil de escalar o sistema inteiro
- ❖ Acessível (respeita configurações do usuário)
- ❖ Perfeita para design systems

Por isso frameworks e libs usam rem como **padrão**.

É a medida relativa **mais usada** para tipografias (fontes).

CSS: em x rem

css tips

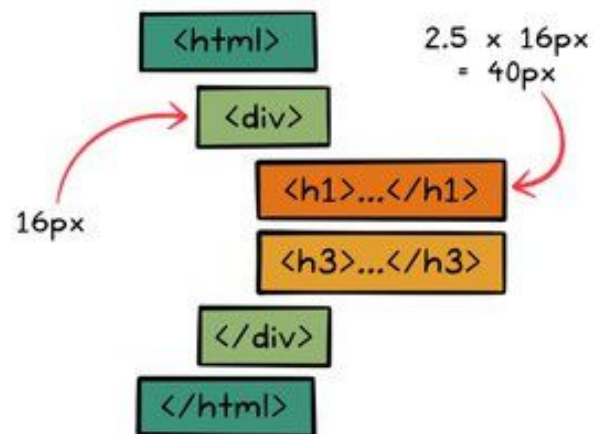
em vs rem

by levelupcoding.co

em

Relative to the font size of the parent element.

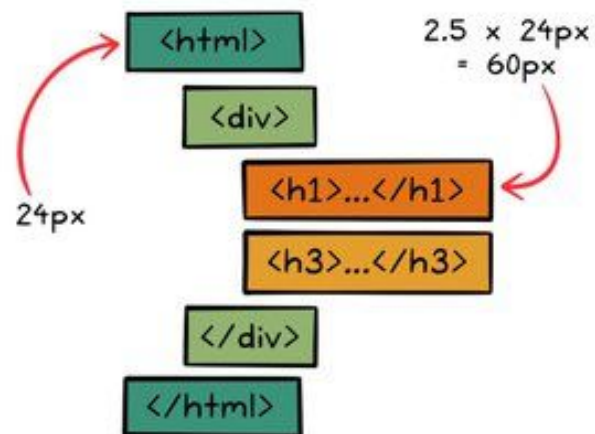
```
h1 { font-size: 2.5em; }
```



rem

Relative to the font size of the root element.

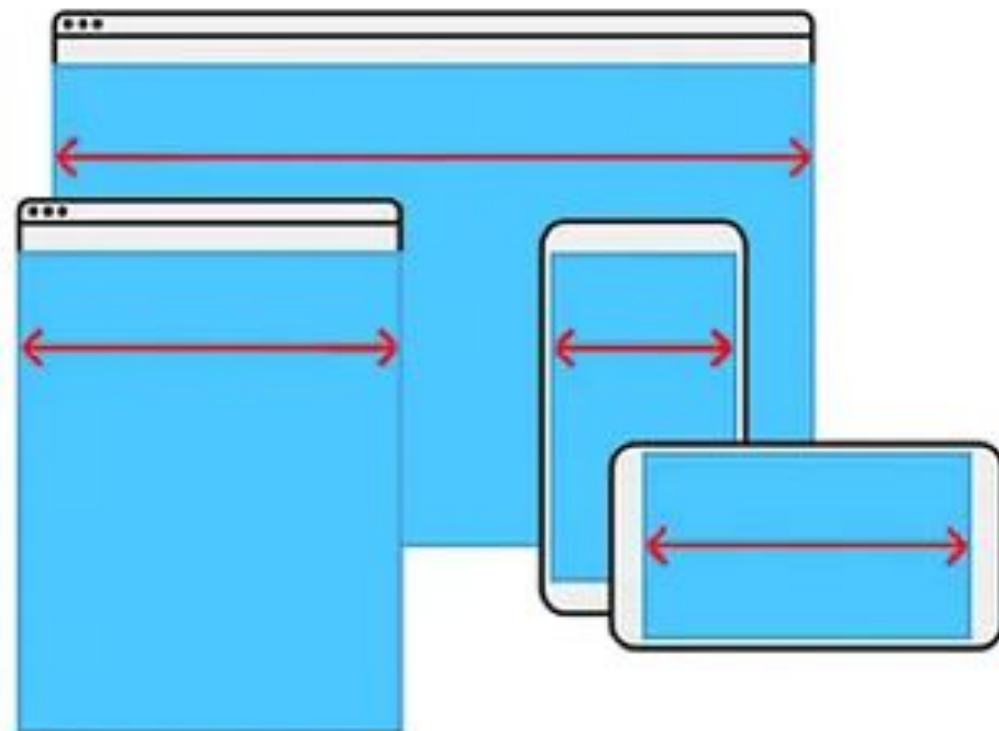
```
h1 { font-size: 2.5rem; }
```



CSS: unidades de medidas flexíveis vw

- ❖ **vw (viewport width):** é relativo a 1% da largura da viewport. É utilizado para definir larguras proporcionais à tela.
- ❖ Pode incluir a largura da barra de rolagem vertical em alguns navegadores, o que pode causar overflow horizontal e rolagem lateral indesejada.
- ❖ Por isso, em muitos casos, `width: 100%` é mais seguro.

VW



CSS: unidades de medidas flexíveis vw

❖ Exemplo vw (viewport width)

Exemplo:

❖ Ocupa toda a largura da viewport.

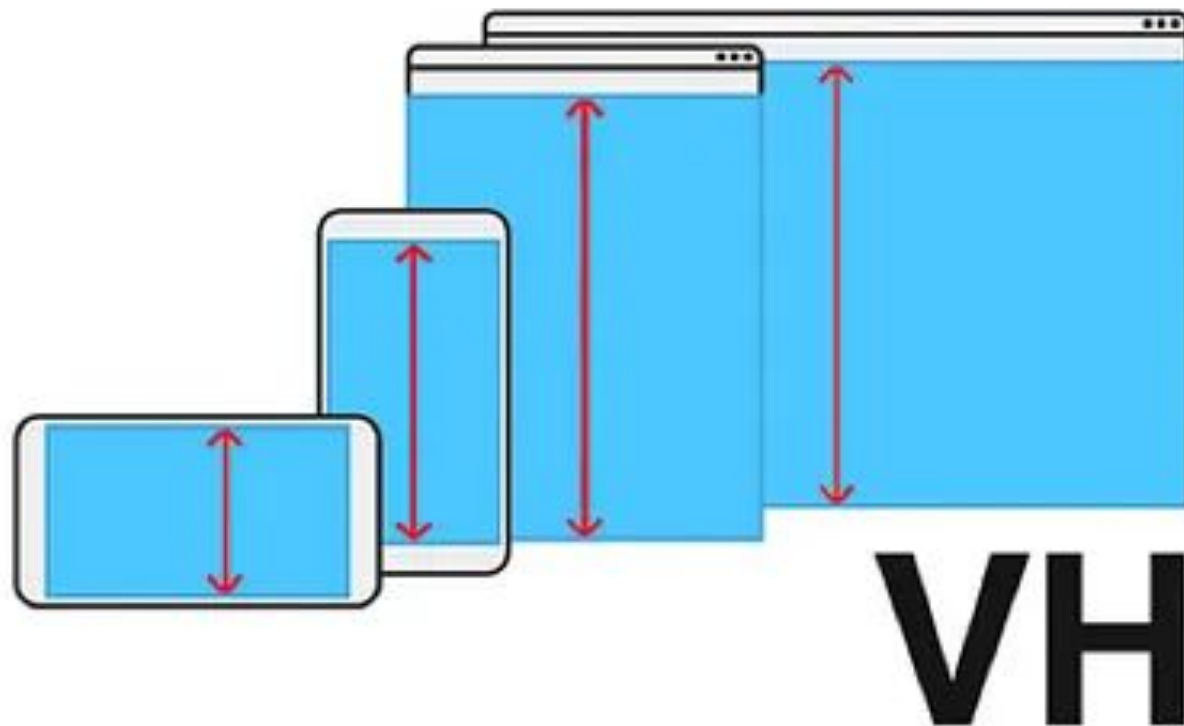
```
main {  
  width: 100vw;  
}
```

Fonte: autora, 2026

CSS: unidades de medidas flexíveis vh

- ❖ **vh (viewport height):** é relativo à altura da viewport.

Entretanto, no mobile, o vh pode considerar a altura total da tela **ignorando elementos dinâmicos** da interface, como barra de endereço, barra inferior e teclado virtual, o que pode causar problemas como conteúdo cortado ou **rolagem inesperada**.



CSS: unidades de medidas flexíveis vh

❖ Exemplo vh (viewport height)

Exemplo:

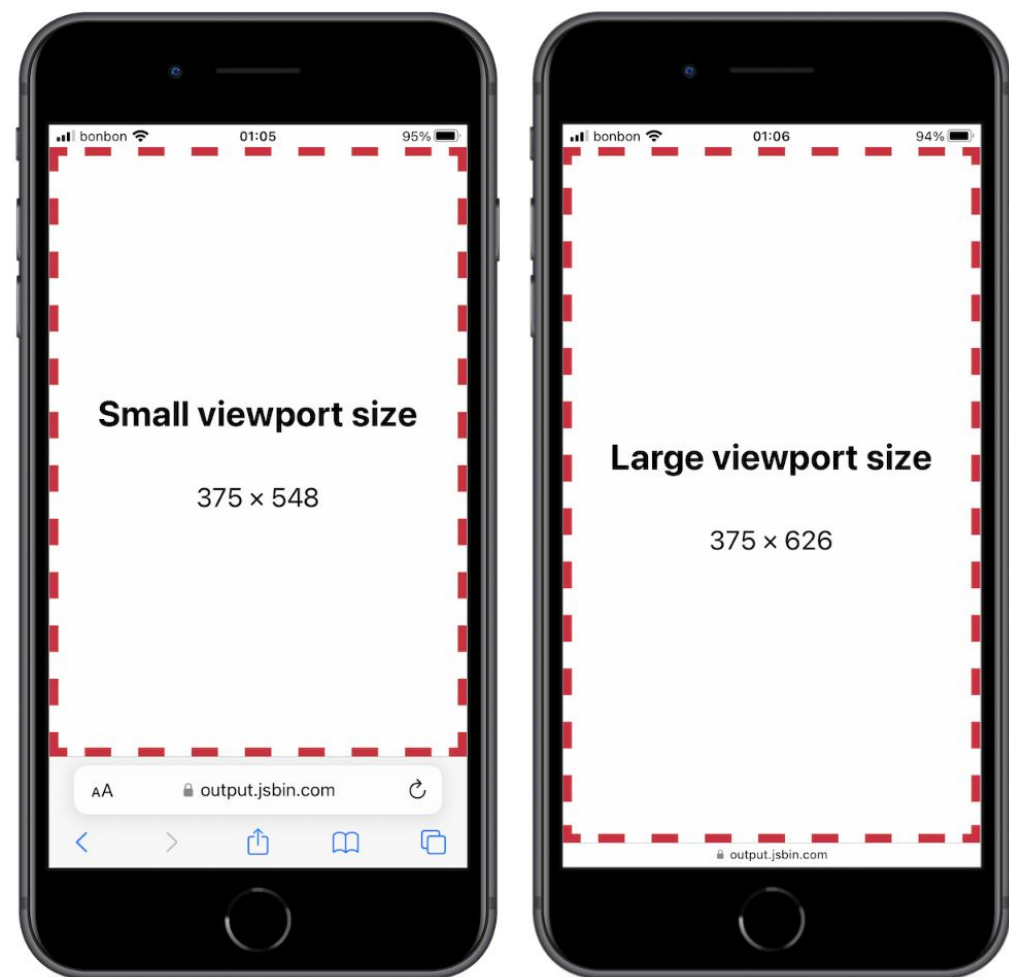
❖ Ocupa toda a altura da viewport.

```
main {  
  height: 100vh;  
}
```

Fonte: autora, 2026

CSS: unidades de medidas flexíveis dvw

- ❖ dvw (dynamic viewport width) é relativo a 1% da largura visível da viewport, atualizada dinamicamente conforme mudanças na interface (ex: barras do navegador).
- ❖ A largura do Viewport quase não altera no mobile, quebras maiores são vistas mais perante a altura.
- ❖ Aqui um problema comumente visto é uma rolagem lateral inesperada.



CSS: unidades de medidas flexíveis dvw

❖ Exemplo de dvw

Exemplo:

- ❖ Ocupa toda a largura visível atual, considerando **mudanças dinâmicas**.

```
main {  
  width: 100dvw;  
}
```

Fonte: autora, 2026

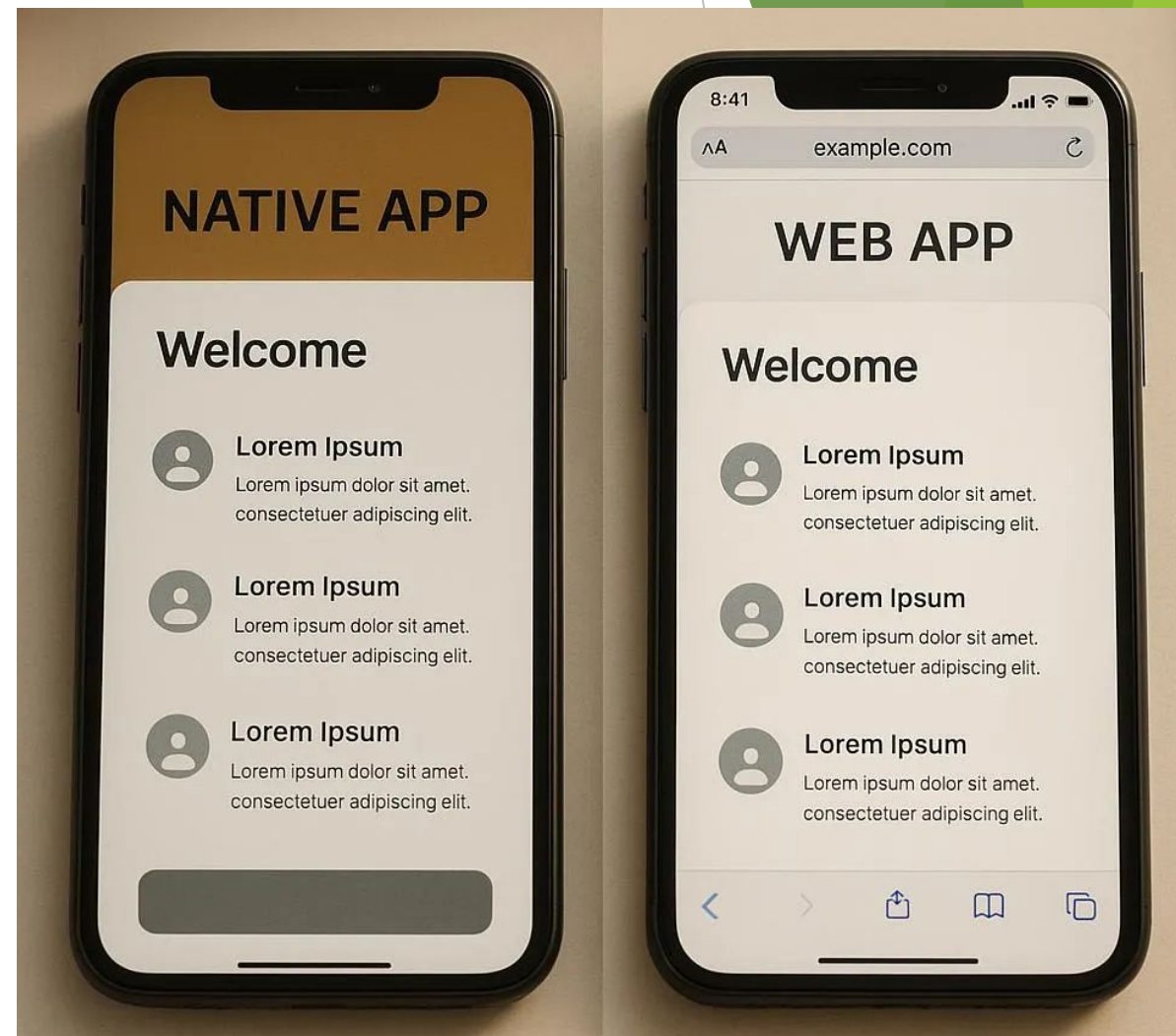
CSS: unidades de medidas flexíveis dvw



CSS: unidades de medidas flexíveis *dvw*

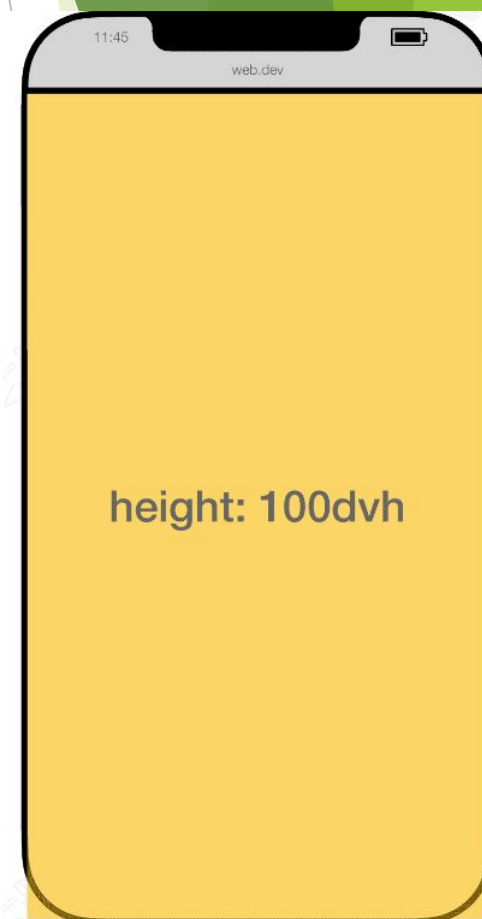
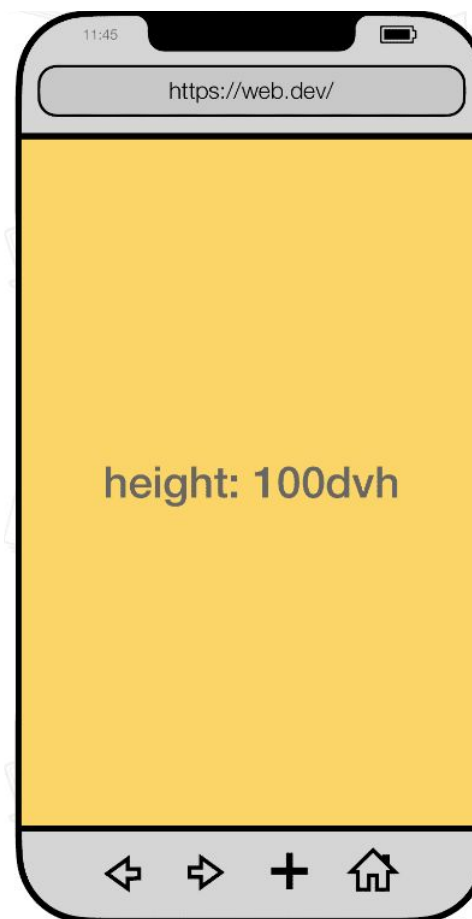
Quando usar o *dvw*?

- ❖ Apps fullscreen muito controlados
- ❖ Layouts que realmente reagem a mudanças de viewport horizontal
- ❖ Ambientes com UI lateral dinâmica (menos comum)



CSS: unidades de medidas flexíveis dvh

- ❖ **dvh (dynamic viewport height):** representa 1% da altura visível da viewport dinâmica, atualizada em tempo real conforme mudanças na interface do navegador (como barra de endereço ou teclado).
- ❖ Essa medida está mais alinhado com o conceito de mobile-first e apps web (PWA) que simulam apps nativos.



CSS: unidades de medidas flexíveis dvh

❖ Exemplo dvh

Exemplo:

- ❖ Ocupa toda a altura real atual da tela, se adaptando dinamicamente.

```
main {  
  height: 100dvh;  
}
```

Fonte: autora, 2026

CSS: como usar as medidas flexíveis?

- ❖ As medidas flexíveis para largura oferecem menos bugs visuais, o que realmente causa problemas são as medidas de altura da tela.
- ❖ Para largura, ainda é muito usado o width com **porcentagem**.
- ❖ Já para altura, é definido regras para ***min-height*** usando svh (menor altura da viewport) e ***height*** usando dvh (dynamic viewport width).

```
main {  
  min-height: 100svh;  
  /* altura mínima possível da tela é indicada como altura mínima do elemento */  
  height: 100dvh;  
  /* a altura do elemento acompanha a altura da viewport */  
  width: 100%;  
}
```

CSS: unidades de medidas flexíveis

- ❖ Ao usar as medidas flexíveis, o layout passa a “respirar” melhor, se adaptando a celulares, tablets e computadores **sem precisar de muitos ajustes manuais.**
- ❖ Isso é essencial hoje em dia, já que cada usuário pode acessar seu site em um **dispositivo diferente.**

CSS imagens adaptativas

CSS: imagens adaptativas com img

Podemos criar imagens que se adaptam a diferentes tamanhos de larguras de tela e são exibidas corretamente. Atuando não somente acessibilidade mais performance.

- ❖ *img* com srcset e sizes permite que o navegador escolha a melhor imagem com base no espaço que ela **ocupará** na tela, na **largura** da viewport e na densidade de **pixels** do dispositivo.
- ❖ A mesma imagem tem **tamanhos diferentes**.

CSS: imagens adaptativas com img

srcset

- ❖ img-400.jpg tem 400px de largura
- ❖ img-800.jpg tem 800px de largura
- ❖ img-1200.jpg tem 1200px de largura

```

```

CSS: imagens adaptativas com img

sizes

- ❖ até 768px: imagem ocupa 100% da tela
- ❖ acima dessa largura em px: ocupa 800px fixos de largura

```

```

CSS: imagens adaptativas com img

O navegador decide qual imagem apresentar analisando:

- ❖ tamanho da tela (viewport, espaço visível)
- ❖ tamanho que a imagem vai ocupar (sizes)
- ❖ resolução do dispositivo (pixels físicos da tela, hardware)

CSS: imagens adaptativas com picture

Permite fornecer **múltiplas versões** de uma imagem para diferentes situações (como tamanho de tela ou formato).

- ❖ **source** define imagens alternativas com condições (media, type)

```
<picture>
  <!-- Mobile -->
  <source
    media="(max-width: 768px)"
    srcset="banner-mobile.jpg"
  >

  <!-- Desktop -->
  <source
    media="(min-width: 769px)"
    srcset="banner-desktop.jpg"
  >

  
</picture>
```

CSS: imagens adaptativas com picture

- ❖ img: é um elemento obrigatório dentro do *picture*.
- ❖ Por definir um valor para atributo *alt* garante a acessibilidade da imagem e serve como fallback caso as outras imagens não possam ser renderizadas.

```
<picture>
  <!-- Mobile -->
  <source
    media="(max-width: 768px)"
    srcset="banner-mobile.jpg"
  >

  <!-- Desktop -->
  <source
    media="(min-width: 769px)"
    srcset="banner-desktop.jpg"
  >

  
</picture>
```


CSS: imagens adaptativas com picture

O navegador decide qual imagem apresentar analisando:

- ❖ Lê os `<source>` de cima para baixo
- ❖ Testa a condição (media, type)
- ❖ Usa o primeiro que for válido
- ❖ Se nenhum for válido vai usar o `<img>`

CSS: estilos para imagens fluidas

- ❖ Ao definir **max-width: 100%**, a imagem ocupará no máximo toda a largura disponível.
- ❖ Enquanto **height: auto** preserva sua proporção original, evitando distorções.
- ❖ O uso de **display: block** remove espaços indesejados e garante melhor controle no layout.
- ❖ Deixar a imagem fluida resolve como ela se comporta no layout, mas não deixa uma imagem 100% compatível com **todos os tipos de telas** nem trata **questões de performance**.

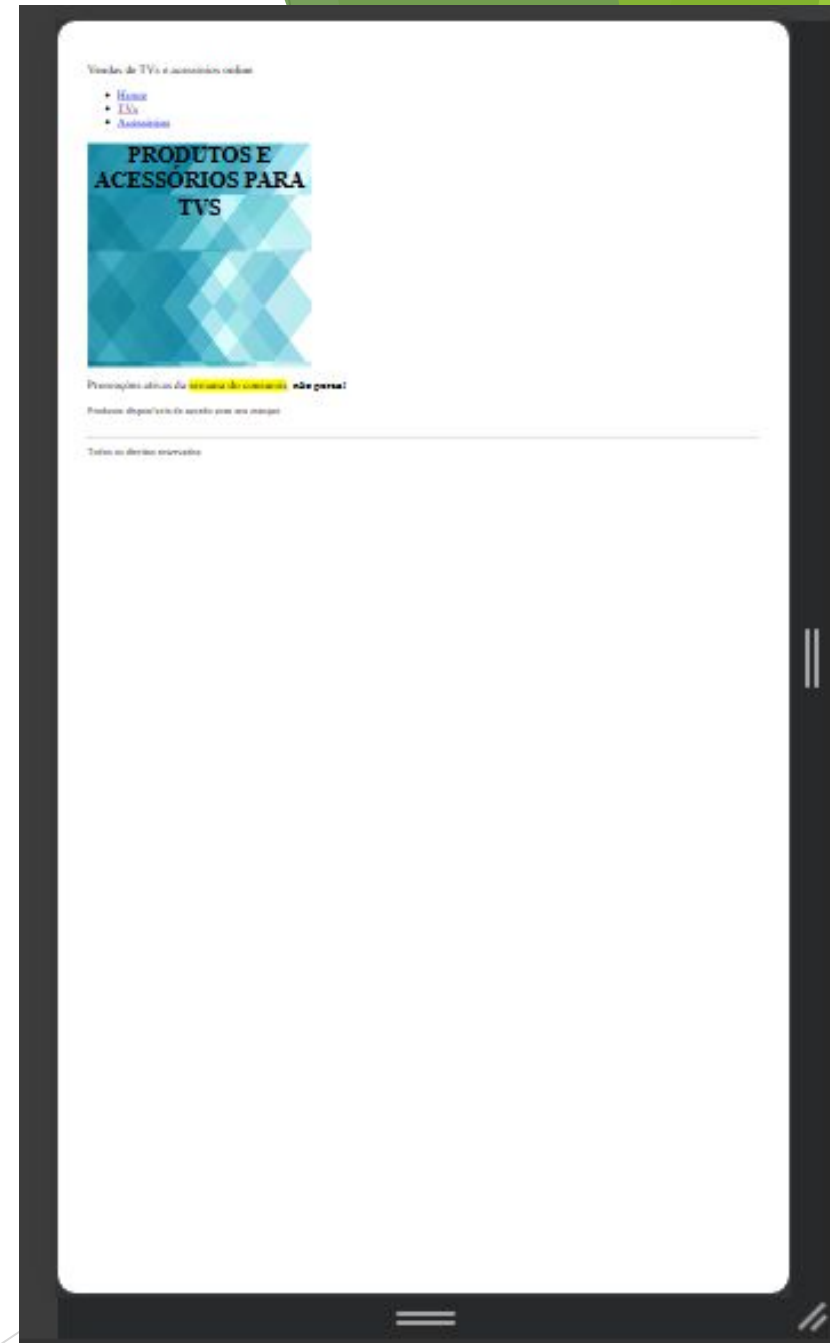
```
.image {  
  display: block;  
  height: auto;  
  max-width: 100%;  
}
```

Fonte: autora, 2026

Meta viewport

HTML: meta viewport

- ❖ É utilizada dentro da tag `<head>` para definir como o conteúdo deve se **adaptar** à largura da tela do dispositivo.
- ❖ Sem essa configuração, o navegador tende a aplicar um zoom automático e renderizar a página como se fosse um **desktop reduzido**, o que faz com que textos fiquem pequenos, elementos desproporcionais e a navegação difícil.



HTML: meta viewport

- ❖ Essa configuração, define que a largura da página deve acompanhar a largura do dispositivo.
- ❖ E que o zoom inicial será de 100%, ou seja, 1.0.

```
<head>  
  <meta charset="UTF-8" />  
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />  
  <title>Vendas de TVs e acessórios online</title>  
  <link rel="stylesheet" href="./styles.css" />  
</head>
```



HTML: meta viewport

- ❖ Outros parâmetros também podem ser utilizados, como maximum-scale, minimum-scale e user-scalable, que controlam o nível de zoom permitido, embora seu uso deva ser feito com cautela para não prejudicar a acessibilidade.

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0,
  minimum-scale=1.0, maximum-scale=5.0, user-scalable=yes">
  <title>Vendas de TVs e acessórios online</title>
  <link rel="stylesheet" href="./styles.css" />
</head>
```

Fonte: autora, 2026



HTML: meta viewport

- ❖ **width=device-width:** usa a largura real do dispositivo
- ❖ **initial-scale=1.0:** zoom inicial padrão (sem distorção)

```
<head>  
  <meta charset="UTF-8" />  
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />  
  <title>Vendas de TVs</title>
```


HTML: meta viewport

- ❖ **minimum-scale=1.0**: evita zoom menor que 100% (não “afasta” demais o conteúdo)
- ❖ **maximum-scale=5.0**: permite dar zoom até 500% (ótimo para acessibilidade)
- ❖ **user-scalable=yes**: permite que o usuário dê zoom manualmente (control ou pinça)

```
<meta charset="UTF-8" />  
<meta name="viewport" content="width=device-width, initial-scale=1.0,  
minimum-scale=1.0, maximum-scale=5.0, user-scalable=yes" />
```

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Dúvidas?

Exercício

CSS: exercício 3

- ❖ all 1: Toque as medidas das tipografias fixas (px) para relativas (rem).

```
<> category_acessorios.html  
<> category_tvs.html  
<> products.html  
# styles.css
```

```
/* 1 */  
* {  
  font-size: 12px;  
}  
  
h1 {  
  font-size: 1.875rem;  
  text-transform: uppercase;  
}  
  
p,  
a,  
ul,  
ol,  
mark,  
strong,  
em,  
span,  
b,  
i {  
  font-size: 1.25rem;  
}  
  
small {  
  font-size: 1rem;  
}
```

CSS: exercício 3

- ❖ ct 1: separe os campos do formulário de solicitação de tvs em duas divs.

```
<section class="products-form">
  <form id="tv-request-form">
    <fieldset>
      <!-- agrupador de campos relacionados, geralmente seguido |
      <legend>Solicitação de Modelo de TV</legend>

      <div class="col">...
      </div>

      <div class="col">...
      </div>

    </fieldset>
  </form>
</section>
```

> images
<> category_acessorios.html
<> category_tvs.html
<> products.html

```
<div class="col">
  <!-- Nome -->
  <label for="nome">Nome:</label>
  <input
    type="text"
    id="nome"
    name="nome"
    placeholder="Ex: Maria Silva"
    autocomplete="name"
    required
  />
  <!-- Email -->
  <label for="email">E-mail:</label>
  <input
    type="email"
    id="email"
    name="email"
    placeholder="Ex: maria@email.com"
    autocomplete="email"
    required
  />
</div>
```

CSS: exercício 3

- ❖ Etapa 2: coloque os botões do formulário numa tag *menu* e adicione a classe *input-model* no input do *datalist*.

```
<section class="products-form">
  <form id="tv-request-form">
    <fieldset>
      <!-- agrupador de campos relacionados, geralmente <div>
      <legend>Solicitação de Modelo de TV</legend>

      <div class="col">...
      </div>

      <div class="col">...
      </div>

    </fieldset>
  </form>
</section>
```

> images
<> category_acessorios.html
<> category_tvs.html
<> products.html

```
<div class="col">
  <!-- Modelo -->
  <label for="modelo">Modelo de TV desejado:</label>
  <input
    class="input-model"
    list="modelos-tv"
    id="modelo"
    name="modelo"
    placeholder="Escolha ou digite um modelo"
    required
  />

  <datalist id="modelos-tv">
    <option value="1">Samsung</option>
    <option value="2">LG</option>
  </datalist>

  <!-- Botões -->
  <menu>
    <button type="reset">Limpar formulário</button>
    <button type="submit">Enviar solicitação</button>
  </menu>
</div>
</fieldset>
```


CSS: exercício 3

- ❖ ct 3: crie regras de estilo para a classe `.col` apresentar 2 itens por linha centralizados.
- ❖ Defina largura para a coluna (width) e alinhados os elementos lado a lado (inline-block)

```
/* 3 */  
.col {  
  width: 49%; /* duas colunas por linha */  
  display: inline-block; /* um elemento do lado do outro */  
}
```

```
<> category_acessories.html  
<> category_tvs.html  
<> products.html  
# styles.css
```

CSS: exercício 3

- ❖ ct 4: apresente a legenda e os campos dos formulários **centralizado** dentro de cada coluna (label, input, text-align, margin).

```
<> category_acessories.html
<> category_tvs.html
<> products.html
# styles.css
```

```
/* 4 */
label,
button {
    text-align: center; /* textos centralizados */
    margin-top: 10px;
}

label,
input {
    width: 80%;
}

label,
input,
.products-form {
    display: block;
    margin-right: auto; /* elementos centralizados */
    margin-left: auto;
}
```

CSS: exercício 3

- ❖ ct 5: defina uma largura máxima para o formulário de produtos não fique com campos gigantes (max-width).

```
/* ct 5 */  
.products-form {  
  max-width: 330px;  
}
```

```
<> category_acessories.html  
<> category_tvs.html  
<> products.html  
# styles.css
```

Fonte: autora, 2026

CSS: exercício 4

❖ category_tvs.html

CATEGORIA DE PRODUTOS

TVs

Lista de TVs disponíveis com preços e parcelamento

Marca	Imagem	Preço	Parcelamento	Detalhes
Samsung		R\$ 2.000,00	R\$ 200,00 em 10x	Ver mais sobre TVs Samsung
LG		R\$ 5.000,00	R\$ 500,00 em 10x	Ver mais sobre TVs LG

Solicitação de Modelo de TV

Modelo de TV desejado:

Nome:

E-mail:

Venda de TVs e acessórios online

- [Home](#)
- [TVs](#)
- [Acessórios](#)

CATEGORIA DE PRODUTOS

TVs

Lista de TVs disponíveis com preços e parcelamento

Marca	Imagem	Preço	Parcelamento	Detalhes
Samsung		R\$ 2.000,00	R\$ 200,00 em 10x	Ver mais sobre TVs Samsung
LG		R\$ 5.000,00	R\$ 500,00 em 10x	Ver mais sobre TVs LG

Solicitação de Modelo de TV

Modelo de TV desejado:

Nome:

E-mail:

Principais dúvidas

► Como solicitar um modelo?
Todos os direitos reservados

Salve esse código no seu email,
Google Drive ou GitHub!

Referência da aula

- ❖ AIGMÜLLER, Lara. Tutorial: The CSS box model & layout basics. Scale, 4 jun. 2022. Disponível em: <https://www.scale.at/blog/css-box-model>. Acesso em: 20 mar. 2026.
- ❖ GODOY, Letícia. Unidade de medida no CSS. Medium, 27 fev. 2024. Disponível em: <https://medium.com/@letgodoy/unidade-de-medida-no-css-089e76b4d947>. Acesso em: 30 mar. 2026.
- ❖ SAN INTERNET. Design responsivo para seu site. San Internet Blog, 7 jan. 2026. Disponível em: <https://blog.saninternet.com/design-responsivo>. Acesso em: 30 mar. 2026

Referência da aula

- ❖ DHAWAN, Sahil. What's new in CSS 3. Medium, 13 maio 2018. Disponível em: <https://medium.com/beginners-guide-to-mobile-web-development/whats-new-in-css-3-dcd7fa6122e1>. Acesso em: 30 mar. 2026.
- ❖ JUDIS, Stefan. The ex unit in CSS. Stefan Judis, 2023. Disponível em: <https://www.stefanjudis.com/today-i-learned/the-ex-unit-in-css/>. Acesso em: 30 mar. 2026
- ❖ VIDAS, Šime. New CSS viewport units do not solve the classic scrollbar problem. Smashing Magazine, 20 dez. 2023. Disponível em: <https://www.smashingmagazine.com/2023/12/new-css-viewport-units-not-solve-classic-scrollbar-problem/>. Acesso em: 30 mar. 2026

Referência da aula

- ❖ MORETTI, Luigi. Unidades CSS: vh, dvh, lvh e svh - entenda o que são e quando usar. Moretti.dev, 13 maio 2025. Disponível em: <https://moretti.dev/unidades-css-vh-dvh-lvh-svh/>. Acesso em: 30 mar. 2026.



univille.br